ADR 0003: pgvector in Postgres instead of Pinecone

Status	Accepted (meeting decision to use pgvector); the scaffold's Pinecone integration is to be replaced
Date	2026-10-19
Related	`04b-database-notes.md` section 3, the scaffold's `catalog/vectors.py` and `integrations/pinecone.py`

Context

Product embeddings are 512 numbers made by the CLIP service. Today they sit in a JSON column and every request scores them in Python, which does not scale. The scaffold plans a separate Pinecone index. The meeting agreed to use Postgres with the pgvector extension. Feed queries are always filtered (approved, in stock, gender, price, category), and product status changes in Postgres.

Options

1. **pgvector** in the same database: one system, filters in SQL, transactional with product status.
2. **Pinecone** (hosted vector database): scales further, but is a second vendor, a cost, and needs a sync job so it never drifts from product status.
3. **Keep JSON and in-process scoring:** simplest, but memory and latency grow with the catalog.

Decision

Option 1.

- Table `product_embeddings(product_id, model_version, embedding vector(512), ...)` with an **HNSW index using cosine distance**.
- User taste lives in `user_taste_clusters` (`vector(512)`), one cluster in the MVP and up to four later.
- The scaffold's Pinecone client and `USE_PINECONE` setting are removed; the vector functions in `catalog/vectors.py` run SQL.
- The MVP slice keeps the JSON column and scores in memory (`06` section 6); pgvector arrives with the new server.

Consequences

- The chosen Postgres host must support the `pgvector` extension; confirm before the first migration.
- Filtered nearest-neighbour recall must be tested as the catalog grows; at a few thousand products an exact scan is acceptable.
- No vector sync job to run and no second bill.

Revisit when

The catalog reaches roughly a million vectors, filtered vector queries miss their latency budget on the database, or a feature needs hybrid text and vector search that Postgres cannot serve well.